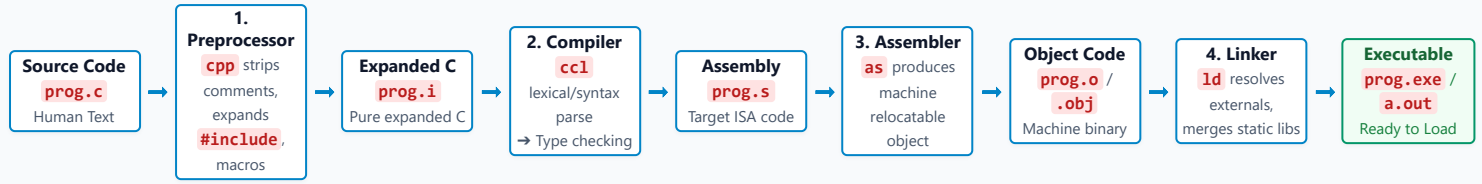


UNIT I: PROGRAMMING FUNDAMENTALS, COMPILATION PIPELINE & LOGIC

Weightage: 25% | Syllabus Section 1

1. THE 4-STAGE C COMPILATION PIPELINE & FILE TRANSFORMATIONS



2. C PROCESS MEMORY LAYOUT (RAM)

Stack Segment (High Memory Local vars, parameters, return addresses → Grows Down !)	
↓ Stack / Heap Free Space Boundary ↑	
Heap Segment (Grows Up !)	Dynamic memory: malloc, calloc, realloc
BSS Segment (Uninitialized Data)	Uninitialized globals & statics (auto-zeroed by OS)
Data Segment (Initialized DS)	Explicitly initialized globals & static variables
Text / Code Segment (Low Memory 0x0)	Binary CPU machine instructions (Read-Only)

Structure of C: 1. Documentation → 2. Link/Preprocessors → 3. Definition → 4. Global Declaration → 5. `main()` → 6. Sub-functions.

3. LOGIC, COMPLEXITY & DEBUGGING

Concept	Definition / Characteristic
Deductive Logic	Top-down general theory → specific conclusion (guaranteed true if premises valid).
Inductive Logic	Bottom-up specific observations → generalized patterns (probabilistic).
Complexity Big-O	$O(1)$ Const $O(\log n)$ Binary Search $O(n)$ Linear Search $O(n^2)$ Bubble Sort.
Black vs White Box	Black-box: Test inputs vs outputs without internal code view. White-box: Branch & path coverage.
Dry Run Tracing	Manual step-by-step table tracking variable states against source line indices.

4. TRANSLATORS & ERROR CLASSIFICATION

Translator	Working Principle
Compiler	Scans entire file at once; generates .obj / .exe; fast execution; reports all errors.
Interpreter	Translates & executes line-by-line; stops on 1st error; slower execution; no object file saved.
Assembler	1-to-1 conversion from mnemonics (e.g. MOV, ADD) directly to binary machine code.

Bug Types:

- Syntax:** Grammar violation (e.g., missing `;`) caught at compile time.
- Linker:** Unresolved symbol (e.g., calling undefined `foo()`).
- Runtime:** Crash during execution (e.g., Divide by 0, SIGSEGV).
- Semantic/Logic:** Runs fine, produces wrong output (e.g., `i <= n` off-by-one).

UNIT II: CORE PROGRAMMING CONCEPTS, OPERATORS & CONTROL STRUCTURES

Weightage: 25% | Syllabus Section 2

1. PRIMITIVE DATA TYPES & SPECIFIERS

Type	Size	Format	Range / Description
char	1 B	%c	-128 to 127 (ASCII standard)
int	2 / 4 B	%d, %i	-2,147,483,648 to 2,147,483,647 (32-bit)
float	4 B	%f	3.4E-38 to 3.4E+38 (6 decimal places)
double	8 B	%lf	1.7E-308 to 1.7E+308 (15 decimal places)
unsigned	4 B	%u	0 to 4,294,967,295 (positive values only)

2. TYPE CONVERSION & CASTING

- 1. Implicit (Promotion):** Automatic type upgrade by compiler to prevent precision loss:
`bool` → `char` → `short` → `int` → `unsigned` → `long` → `float` → `double` → `long double`
Hazard: `int a=5, b=2; float c = a/b;` yields `2.0` because integer division occurs before assignment!
- 2. Explicit (Typecasting):** Programmer forced:
`float c = (float)a / b;` yields `2.5`.

3. STANDARD HEADER FILES TAXONOMY

Header	Core Functions / Purpose
<stdio.h>	printf, scanf, fopen, fclose, fprintf, fclose
<stdlib.h>	malloc, calloc, realloc, free, exit, atoi
<string.h>	strlen, strcpy, strcat, strcmp, strrev
<math.h>	pow, sqrt, sin, cos, floor, ceil, fabs
<ctype.h>	isalpha, isdigit, toupper, tolower

4. COMPLETE 9-TIER OPERATOR PRECEDENCE & ASSOCIATIVITY TABLE

Tier	Category	Operators	Associativity	Key Exam Rule / Behavior
1 (Highest)	Postfix / Member	() [] -> . ++ --	Left-to-Right	Parentheses, array subscript, struct dereference evaluated first.
2	Unary	+ - ! ~ ++ -- (type) * & sizeof	Right-to-Left	Prefix inc/dec, address-of &, dereference *, explicit cast.
3	Multiplicative	* / %	Left-to-Right	Modulus % operates on integers only (illegal with float/double).
4	Additive	+ -	Left-to-Right	Addition and subtraction arithmetic.
5	Relational	< <= > >=	Left-to-Right	Yields integer 1 (true) or 0 (false).
6	Equality	== !=	Left-to-Right	Lower precedence than relational: <code>a < b == c</code> evaluates <code>(a < b) == c</code> .
7	Logical	&& (AND) then (OR)	Left-to-Right	Short-circuit: In <code>A && B</code> , if A=0, B skipped! In <code>A B</code> , if A=1, B skipped!
8	Ternary (Conditional)	? :	Right-to-Left	<code>condition ? expr:true : expr:false</code>
9 (Lowest)	Assignment & Comma	= += -= *= /= %= ,	R-to-L (=) L-to-R (,)	Comma operator evaluates expressions and returns the rightmost result.

5. DECISION MAKING STRUCTURES & SWITCH FALLTHROUGH

If - Else If Ladder:

Condition 1?
True → Block 1 False → Next Cond
Condition 2?
All false → Default else block executed.

Switch-Case Anatomy:

- Expression must evaluate to **integral/char** constant.
- `break;` terminates execution and jumps outside the block.
- Fall-through Hazard:** If `break;` is omitted, execution cascades sequentially into every subsequent case until a `break` or `}`!

6. LOOP LIFECYCLE & JUMP STATEMENTS (BREAK, CONTINUE, GOTO)

1. Init <code>i = 0;</code> executed once	2. Condition <code>i < N</code> (Entry-check)	3. Loop Body Code statements execute	4. Update <code>i++</code> loops to step 2
Statement	Action on Execution	Loop State	
<code>break;</code>	Instantly aborts the enclosing loop/switch	Control jumps to statement <i>after</i> loop	
<code>continue;</code>	Skips remaining statements of <i>current</i> pass	Jumps directly to next iteration increment	
<code>goto label;</code>	Unconditional jump directly to <code>label:</code>	Bypasses structured flow (creates spaghetti code)	

UNIT III: ARRAYS, STRINGS, FUNCTIONS & MODULAR PROGRAMMING

Weightage: 25% | Syllabus Section 3

1. ARRAY MEMORY MAPPING & ROW-MAJOR FORMULA

- Contiguous Allocation: Elements stored back-to-back in RAM.
- 1D Array Formula: $\text{Addr}(A[i]) = \text{Base} + (i * \text{sizeof}(\text{type}))$
- 2D Array Row-Major Order (C Standard): Rows stored sequentially in memory.

$$\text{Address}(A[i][j]) = \text{BaseAddress} + (i * C + j) * \text{Size_of_Element}$$

Example: `int A[3][4]` with `Base=1000`, `sizeof(int)=4`. Address of `A[2][1]`:
 $\text{Addr} = 1000 + (2 * 4 + 1) * 4 = 1000 + (9 * 4) = 1036$.

2. SEARCHING & SORTING ALGORITHMS SUMMARY

Algorithm	Best Time	Worst Time	Space	Working Principle / Logic
Linear Search	$O(1)$	$O(n)$	$O(1)$	Sequentially compares target with elements from index 0 to n-1.
Binary Search	$O(1)$	$O(\log n)$	$O(1)$	Requires sorted array; computes $\text{mid} = (\text{low} + \text{high}) / 2$; halves search space.
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$	Repeatedly swaps adjacent elements if $a[j] > a[j+1]$. Sinks max to end each pass.

3. EXHAUSTIVE 12-POINT MASTER STORAGE CLASSES MATRIX (AUTO, REGISTER, STATIC, EXTERN)

Storage Class	Keyword	Physical Storage	Default Value	Scope / Visibility	Lifetime	Key Behavior / Best Use
Automatic	<code>auto</code>	Stack Memory	Garbage value	Local to enclosing block	Function invocation lifetime	Default for all local variables. Freed upon return.
Register	<code>register</code>	CPU Registers	Garbage value	Local to enclosing block	Function invocation lifetime	Speed optimization for loop counters. <code>&regvar</code> address-of operator is ILLEGAL!
Static	<code>static</code>	Data Segment (BSS)	Zero (0 / <code>''</code>)	Local to block or file	Entire program execution	Retains value across repeated function calls. Initialized only once.
External	<code>extern</code>	Data Segment	Zero (0 / <code>''</code>)	Global across all files	Entire program execution	References a global variable defined in another file. No memory allocated in declaration.

4. STRING HANDLING (<STRING.H>)

- Strings are null-terminated (`''` ASCII 0) char arrays.
- `strlen(s)`: Returns char count excluding `''`.
 - `strcpy(dest, src)`: Copies src to dest.
 - `strcat(dest, src)`: Appends src to dest.
 - `strcmp(s1, s2)`: 0 if equal, <0 if `s1 < s2`, >0 if `s1 > s2`.
 - `strrev(s)`: Reverses string in-place.

5. CALL BY VALUE VS REFERENCE

- Call by Value:** Copy of argument is passed. Changes to formal params do not reflect in caller.
- `void swap(int a, int b) { ... }`
- Call by Reference:** Memory address passed via pointers. Modifies actual caller variable.
- `void swap(int *a, int *b) { int t=*a; *a=*b; *b=t; }`
- Called with: `swap(&x, &y);`

6. RECURSION & CALL STACK FRAMES

- A function calling itself requires a **Base Condition** to terminate unwinding, otherwise *Stack Overflow* occurs!
- Factorial Stack Trace:**
- `fact(3) → 3 * fact(2)` (Pushed)
- `fact(2) → 2 * fact(1)` (Pushed)
- `fact(1) → 1` (Base hit, Unwinds → 2 → 6).

UNIT IV: POINTERS, STRUCTURES, UNIONS, DMA & FILE HANDLING

Weightage: 25% | Syllabus Section 4

1. POINTER TAXONOMY: WILD, DANGLING, VOID & FUNCTION POINTERS

Pointer Type	Syntax & Definition	Hazard / Behavior
Wild Pointer	<code>int *ptr;</code> (Uninitialized)	Points to arbitrary memory; dereferencing causes crash! Fix: Initialize to <code>NULL</code> .
Dangling Pointer	<code>free(ptr);</code> (Memory deleted)	Points to freed deallocated memory. Fix: Set <code>ptr = NULL</code> ; immediately after <code>free()</code> .
Generic (Void*)	<code>void *vp = &x;</code>	Generic pointer. Cannot be directly dereferenced; must typecast: <code>*(int*)vp</code> .
Function Pointer	<code>int (*fp)(int, int) = &add;</code>	Holds starting address of executable code in text segment. Called: <code>fp(4, 5)</code> .
Ptr to Array vs Array of Ptrs	<code>int (*p)[5];</code> (Ptr to 5-elem array) <code>int *p[5];</code> (Array of 5 int pointers)	Precedence: <code>[]</code> binds tighter than <code>*</code> without parentheses!

2. REAL-MODE POINTER ARCHITECTURE: NEAR, FAR, & HUGE POINTERS

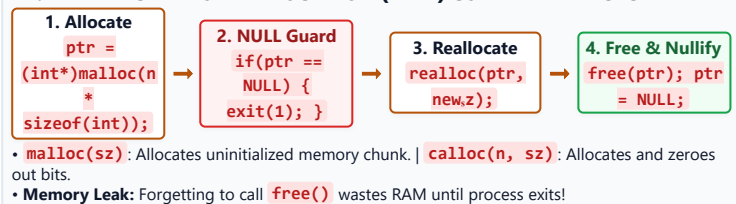
Pointer Type	Size	Address Scheme	Range & Characteristics
Near Pointer	16-bit (2 B)	16-bit Offset only	Limited to 64 KB current default segment (DS/CS). Fast arithmetic.
Far Pointer	32-bit (4 B)	16-bit Segment : 16-bit Offset	Accesses full 1 MB 8086 RAM. Comparison issues because addresses are unnormalized.
Huge Pointer	32-bit (4 B)	Normalized Segment : Offset	Auto-normalizes segment arithmetic on every increment. Reliable comparison across segments.

* Note: In modern 32-bit/64-bit flat memory models, pointers are unified 4-byte or 8-byte virtual memory addresses.

3. STRUCTURE VS UNION MEMORY ARCHITECTURE

Feature	Structure ('struct')	Union ('union')
Keyword	<code>struct Data { int x; char c; };</code>	<code>union Data { int x; char c; };</code>
Total Size	Sum of members + padding (e.g., 4 + 1 + 3 = 8 B)	Size of largest member only (e.g., <code>max(4, 1) = 4 B</code>)
Memory Sharing	Distinct memory location for each member	All members share the <i>exact same</i> starting address
Active Members	All members accessible simultaneously	Only ONE member can hold valid data at any instant
Member Access	<code>obj.member</code> (Direct) <code>ptr->member</code> (Indirect)	<code>u.member</code> (Direct) <code>u_ptr->member</code> (Indirect)

4. DYNAMIC MEMORY ALLOCATION (DMA) COMPLETE LIFECYCLE



5. C FILE HANDLING OPERATIONS & STREAM MODES

Mode	Stream Meaning	File Not Found Behavior	File Already Exists Behavior
"r"	Read Text Stream	Returns <code>NULL</code> error pointer	Opens existing file; pointer at byte 0
"w"	Write Text Stream	Creates a new empty file	Truncates / overwrites file to 0 bytes!
"a"	Append Text Stream	Creates a new empty file	Preserves data; all writes appended at EOF
"r+" / "w+"	Read & Write Mode	<code>r+</code> : Error if absent <code>w+</code> : Creates new	<code>r+</code> : In-place modify <code>w+</code> : Truncates completely
"rb" / "wb"	Binary Stream	Opens file for raw binary structures	No newline translation; use <code>fread()</code> / <code>fwrite()</code>

File API: `FILE *fp = fopen("data.txt", "w");` `fprintf(fp, "%d %s", id, name);` `fscanf(fp, "%d %s", &id, name);` `fgetc(fp) / fputc(c, fp)` `fclose(fp);`